

PROGRAMMING- the process of instructing a computer to solve a problem.

PRO.LANGUAGE- A set of instruction and syntax used to create software

GENERATIONS OF PRO.LANGUAGE

- **1st Generation (1GL):**

Machine code (0s and 1s)

Very fast, hard to write

- **2nd Generation (2GL):**

Assembly language (e.g., MOV, ADD)

Easier than machine code

- **3rd Generation (3GL):**

close to human lang

easier to read, portable

Example: C,C

- **4th Generation (4GL):**

Very high-level, used for specific tasks

Example: SQL, MATLAB

- **5th Generation (5GL):**

Based on logic and AI

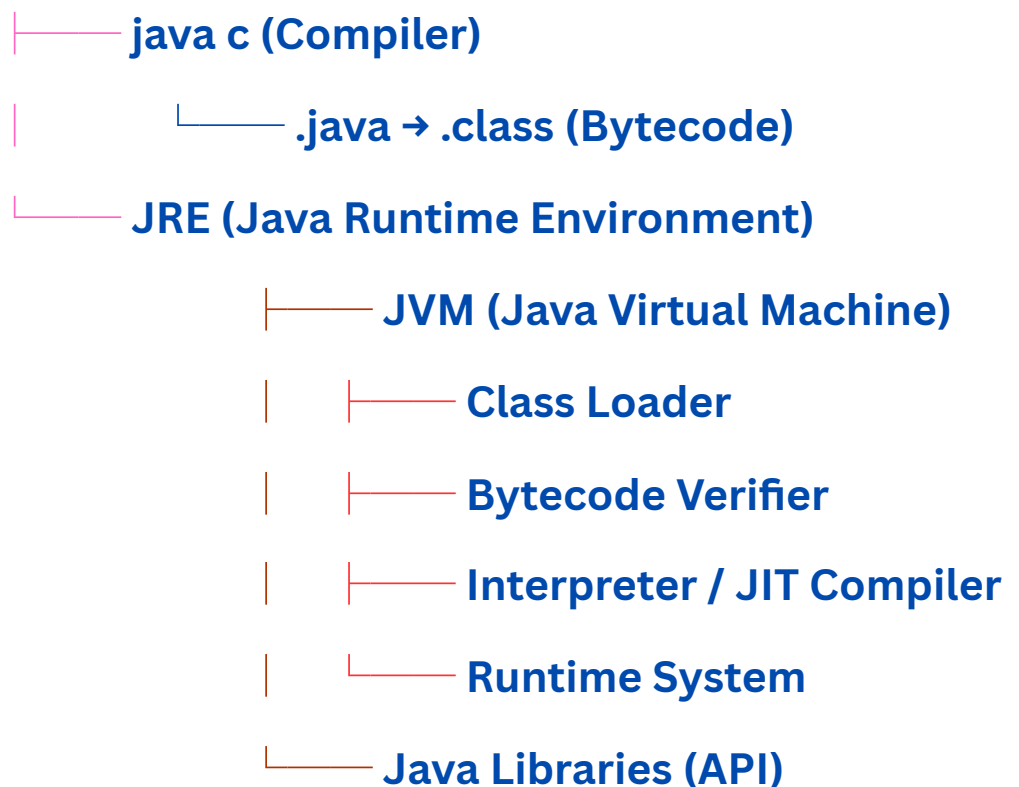
Example: Prolog, LISP

JAVA : high-level, general-purpose, object-oriented, memory-safe programming language

LTS(LONG TERM-SUPPORT): Software version that gets updates ,security patches and bug fixes for a long period.

JAVA SE(STANDARD EDITION): provides core functionality and libraries to build desktop, server, and embedded app.

JDK (Java Development Kit)



JDK (JAVA DEVELOPMENT KIT) : A software package with tools to develop, compile and run Java programs.

JVM (JAVA VIRTUAL MACHINE): Runs Java bytecode and makes Java platform independent .It can run on any system

JAVA BASICS ---

JAVA COMMENTS: note in the code ignored by compiler

```
// single-line
```

```
/* */ multi- line
```

```
/** */ doc comment
```

JAVA LITERALS: data used for representing fixed value

type of literals

- * **string** - a sequence of characters enclosed in double quotes

- * **integer** - whole numbers

 - Decimal (Base 10) -123, 0, -45

 - Binary (Base 2) -0b1010

 - Octal (Base 8) -075

 - Hexadecimal (Base 16) - 0x1A3

- * **floating point** - number with decimal

- * **boolean** - true & false

- * **character** - a single character enclosed in single quotes

- * **null literal** – represents no value

ESCAPE CHARACTERS: Special codes starting with \ for new lines, tabs, quotes, etc.

VARIABLE: A named storage location that holds a value

```
int age = 50
```

(int) - **Variable Type**

(age) - **Variable Name**

(50) - **Assigned Value**

RULES OF VARIABLE

- Use letters (A–Z, a–z)

digits (0–9), _ or \$

- Valid: name, Invalid: 1name

→ Reason: **can't start with a digit**

- Valid: firstName, Invalid: first name

→ Reason: **no spaces allowed.**

- age and Age are different variables

→ Reason: **Java is case-sensitive.(name # Name)**

- Invalid: int, class

→ Reason: **can't use java keywords**

DATA VARIABLE - decide use and scope.

DATA TYPE - show what kind of data variables hold.

PRIMITIVE DATA TYPES		NON-PRIMITIVE DATA TYPES	
non numeric	numeric	string	array
-boolean	-integer		
-char	short		
	int		
	byte		
	long		
	-floating point		
	float		
	double		

Primitive Data Types: Basic types that store one value, like a number, letter, or true/false.

Numeric Data Types: used to store numbers

Widening Type Conversion with Numeric Data Type

byte → short → int → long → float → double

Narrowing Type Conversion with Numeric Data Types

byte ← short ← int ← long ← float ← double

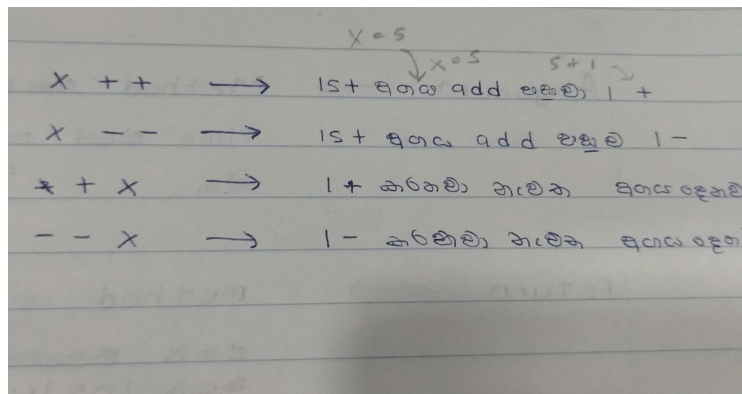
Non-Primitive Data Types: store multiple or complex values like arrays or objects

Arithmetic operators

++ increment ; -- decrement ;

post-condition (x++ / x--)

pre-condition (++x / --x)



Java flow controllers

- conditions ,switch-case statement (for)
- loops for while loops, do-while loops
- jumps-continues, break, return

Arrays

Store multiple values of the same type in one variable.

-Multidimensional Array:

- essentially an array of arrays.

Java Array Creation

Array initializer

Array creation expression

- Array creation expression Only
- Array creation expression with initializer

-Array Initializer:

- Shorthand array initialization is called an Array Initializer

```
int[] y = {1, 2, 3, 4, 5};  
String[] colors = {"Red", "Green", "Blue"}
```

-Arrays.toString():

- print the contents of an array as a readable string.

```
int[] numbers = {1, 2, 3, 4, 5};  
System.out.println(numbers); // Prints something like: [I@6d06d69c (memory address)
```

-Length in Java Arrays:

- how many elements are in an array.

```
int[] numbers = {10, 20, 30, 40, 50};  
System.out.println(numbers.length); // Output: 5
```

-Array Creation Expression:

- This is when you use the new keyword to directly create an array.

```
int[] x = new int[5];  
String[] names = new String[3];
```

-Array Creation Expression With Initializer:

- New keyword, and at the same time, you provide the initial values for the array using {}.

```
int[] x = new int[] {1, 2, 3, 4, 5};
```

```
String[] names = new String[] {"Hi", "Hello", "WOW"};
```

-Update Array Items:

```
int[] numbers = {10, 20, 30, 40};
```

```
numbers[1] = 200;
```

-Existing array assign to another variable

```
int[] x = {10, 20, 30, 40};
```

```
int[] y = x;
```

Methods

code blocks that perform specific tasks

Types of Java Methods



JVM Memory

-Heap Area

-Stack Area -Thread Stack

-Stack Frame_(1,2,3,4,..)

-Class Area

-PC Registry

-Native Method Stack